ANSIBLE

WHY ANSIBLE?

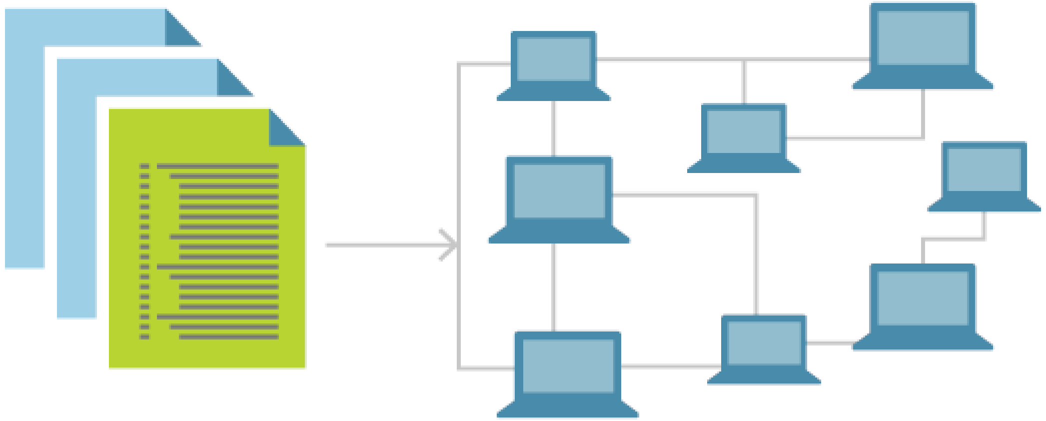
WE NEED A WAY TO
CONFIGURE
MULTIPLE VMS

BASH SCRIPTS
WON'T CUT IT

CONFIGURATION
DRIFT

INFRASTRUCTURE AS CODE

INFRASTRUCTURE AS CODE (IAC) IS A TYPE OF IT INFRASTRUCTURE THAT OPERATIONS TEAMS CAN AUTOMATICALLY MANAGE AND PROVISION THROUGH CODE, RATHER THAN USING A MANUAL PROCESS.



TOOLS

CFEngine

puppet labs

PalletOps

ANSIBLE

CloudFormation

CHEF™

HashiCorp Terraform

SALTSTACK

TOOLS

CONFIGURATION ORCHESTRATION

AUTOMATE THE DEPLOYMENT OF SERVERS AND OTHER INFRASTRUCTURE.

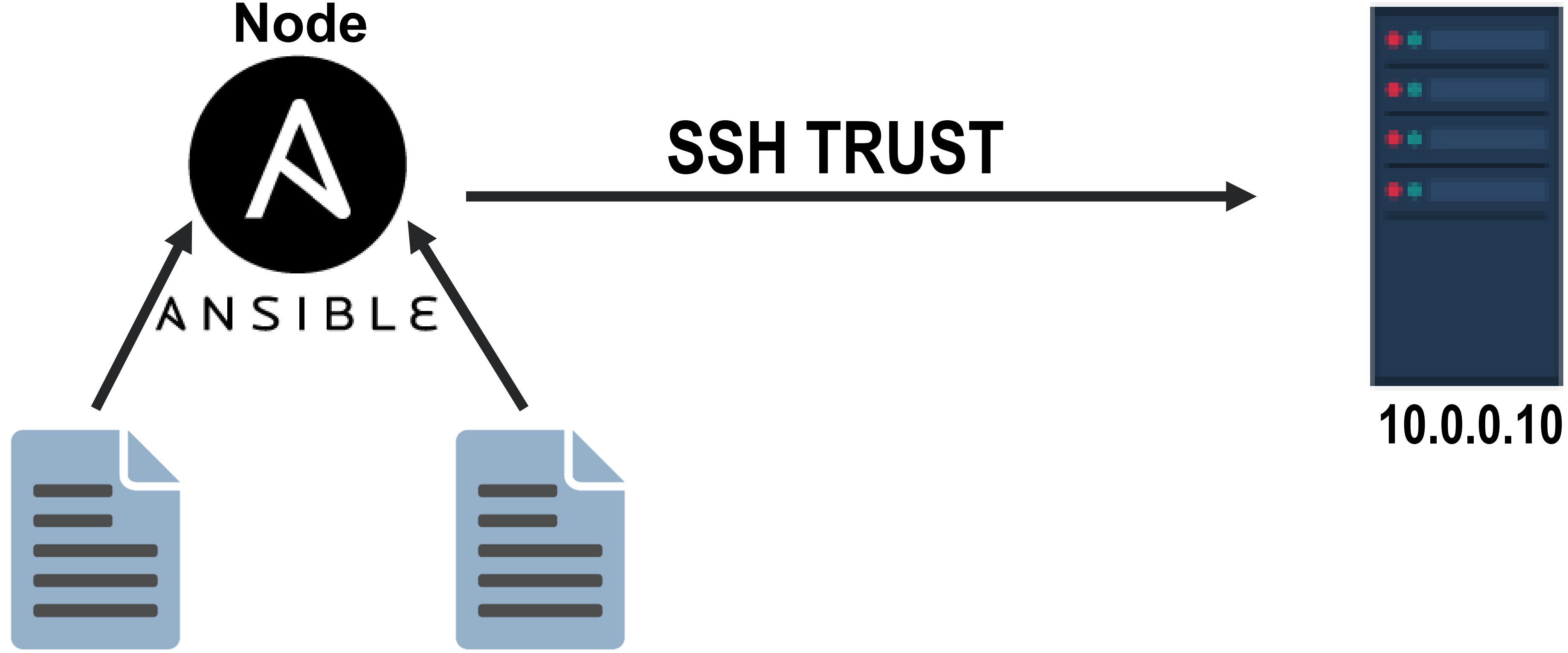
CONFIGURATION MANAGEMENT

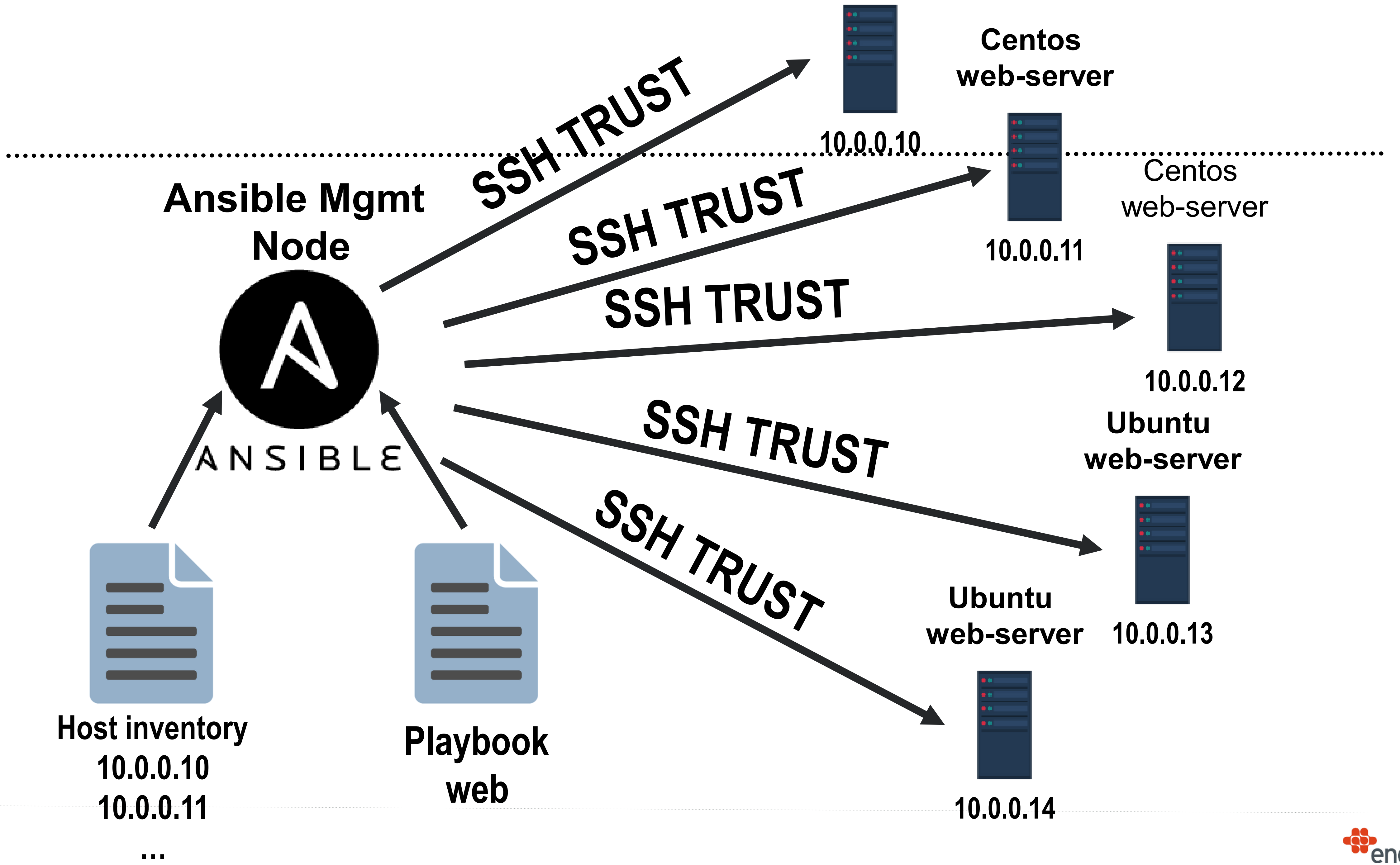
HELP CONFIGURE THE SOFTWARE AND SYSTEMS ON THIS INFRASTRUCTURE

ANSIBLE

- YAML IN THE FORM OF ANSIBLE PLAYBOOKS
- PLAYBOOKS USED FOR GROUPING INSTRUCTIONS
- AGENTLESS MODEL FOR DEPLOYMENT SSH
- TEMPLATING MAKES CONF FILES A BREEZE

OVERVIEW


Ansible Mgmt
Node
ANSIBLE
SSH TRUST
Ubuntu
web-server
10.0.0.10
Host inventory
10.0.0.10
Playbook
web



Ansible Mgmt Node
ANSIBLE
SSH TRUST
SSH TRUST
SSH TRUST
SSH TRUST
SSH TRUST
Centos web-server
10.0.0.10
10.0.0.11
Centos web-server
10.0.0.12
Ubuntu web-server
10.0.0.13
Ubuntu web-server
10.0.0.14
Host inventory
10.0.0.10
10.0.0.11
...
Playbook
web

INVENTORY FILES

DEFINES THE HOSTS,GROUPS OF HOSTS AND VARIABLES

```ini
[webapp]
192.168.60.4
192.168.60.5

[database]
192.168.50.1
192.168.50.2

[switch]
192.168.40.3
192.168.40.4

[tomcat:children]
webapp
switch
```

```yaml
---
all:
  children:
    webapp:
      hosts:
        192.168.60.4:
        192.168.60.5:
    database:
      hosts:
        192.168.50.1:
        192.168.50.2:
    switch:
      hosts:
        192.168.40.3:
        192.168.40.4:
    tomcat:
      children:
        webapp:
        switch:
```

```yaml
---
all:
  children:
    tomcat:
      children:
        webapp:
          hosts:
            192.168.60.4:
            192.168.60.5:
        switch:
          hosts:
            192.168.40.3:
            192.168.40.4:
    database:
      hosts:
        192.168.50.1:
        192.168.50.2:
```

Two types of inventory: INI and YAML

Two default groups: all and ungrouped

Inventory commands:

```
ansible-inventory [-i INVENTORY] --graph
ansible-inventory [-i INVENTORY] --list -y
```

```yaml
---
- hosts: all
  become: yes
  gather_facts: no
  tasks:
    - name: check group exists
      group:
        name: "ansible"
        state: present

    - name: Allow passwordless sudo for group
      lineinfile:
        dest: /etc/sudoers
        state: present
        regexp: ^%ansible
        line: "%ansible ALL=(ALL) NOPASSWD: ALL"
        validate: 'visudo -cf %s'

    - name: Add user ansible
      user:
        name: ansible
        groups: "ansible"
        append: yes
        state: present
        createhome: yes

    - name: install ssh key
      authorized_key:
        user: ansible
        key: "{{ lookup('file', '/tmp/ansible.pvt') }}"

- hosts: webapp
  become: yes
  gather_facts: no
  tasks:
    - name: Install apache tomcat
      yum:
        name: httpd
        state: latest

- hosts: database
  gather_facts: no
  become: yes
  tasks:
    - name: yum install mysql-server rpm
      yum:
        name: https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm
        state: present
```

PLAYBOOKS

TO-DO LIST FOR ANSIBLE THAT CONTAINS A LIST OF TASKS

`ansible-playbook [-i INVENTORY] playbook.yml`

One or more Ansible tasks can be combined to make a play.
Two or more plays can be combined to create an Ansible playbook

TASKS

SET OF ACTIONS (MODULES + ARGS) TO BE EXECUTED

```yaml
- name: Allow passwordless sudo for group
  lineinfile:
    dest: /etc/sudoers
    state: present
    regexp: ^%ansible
    line: "%ansible ALL=(ALL) NOPASSWD: ALL"
    validate: 'visudo -cf %s'
```

VARS

DEFINING VARIABLES:
- INVENTORY (GRUP_VARS,HOST_VARS)
- PLAYBOOK
- GROUP_VARS/HOST_VARS DIRECTORIES
- VARS_FILES
- TASK VARIABLES
- EXTRA VARIABLES

INVENTORY VARS EXAMPLE

```yaml
---
all:
  vars:
    group: ansible
  children:
    tomcat:
      children:
        webapp:
          hosts:
            192.168.60.4:
              host_var1: value
            192.168.60.5:
              host_var2: value
          vars:
            httpd_version: 2.4.46
        switch:
          hosts:
            192.168.40.3:
              host_var3: value
            192.168.40.4:
              host_var4: value
    database:
      hosts:
       192.168.50.1:
       192.168.50.2:
      vars:
        mysql_url_rpm: https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm
        mysql_package_name: mysql-server
```

```ini
[webapp]
192.168.60.4 host_var1: value
192.168.60.5 host_var2: value

[database]
192.168.50.1
192.168.50.2

[switch]
192.168.40.3 host_var3: value
192.168.40.4 host_var4: value

[tomcat:children]
webapp
switch

[all:vars]
group=ansible

[webapp:vars]
httpd_version=2.4.46

[database:vars]
mysql_url_rpm=https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm
mysql_package_name=mysql-server
```

GROUP AND HOST VARS EXAMPLE

Ansible loads host and group variable files by searching paths relative to the inventory file or the playbook file.

Directory layout

```
├── ansible.cfg
├── group_vars
│   ├── all.yml
│   ├── database.yml
│   ├── switch.yml
│   ├── tomcat.yml
│   └── webapp.yml
├── host_vars
│   ├── 192.168.50.1.yml
│   ├── 192.168.50.2.yml
│   ├── 192.168.60.4.yml
│   └── 192.168.60.5.yml
├── internship-playbook.yml
├── inventory
└── inventory.yml
```

Files content

```
[vagrant@localhost group_host_vars]$ cat group_vars/all.yml

group: ansible
[vagrant@localhost group_host_vars]$
[vagrant@localhost group_host_vars]$ cat group_vars/database.yml

mysql_url_rpm: https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm
mysql_package_name: mysql-server
[vagrant@localhost group_host_vars]$
[vagrant@localhost group_host_vars]$ cat group_vars/webapp.yml

httpd_version: 2.4.46
```

PLAYBOOK VARS EXAMPLE

```yaml
---
- hosts: all
  become: yes
  gather_facts: no
  vars:
    group: ansible
  tasks:
    - name: check group exists
      group:
        name: "{{ group }}"
        state: present

    - name: Allow passwordless sudo for group
      lineinfile:
        dest: /etc/sudoers
        state: present
        regexp: ^%ansible
        line: "%{{ group }} ALL=(ALL) NOPASSWD: ALL"
        validate: 'visudo -cf %s'

    - name: Add user ansible
      user:
        name: ansible
        groups: "{{ group }}"
        append: yes
        state: present
        createhome: yes

    - name: install ssh key
      authorized_key:
        user: ansible
        key: "{{ lookup('file', '/tmp/{{ group }}.pvt') }}"

- hosts: webapp
  become: yes
  gather_facts: no
  vars:
    httpd_version: 2.4.46
  tasks:
    - name: Install apache tomcat
      yum:
        name: "httpd-{{ httpd_version }}"
        state: present

- hosts: database
  gather_facts: no
  become: yes
  vars:
    mysql_url_rpm: https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm
    mysql_package_name: mysql-server
  tasks:
    - name: yum install mysql-server rpm
      yum:
        name: "{{ mysql_url_rpm }}"
        state: present

    - name: yum install mysql-server package
      yum:
        name: "{{ mysql_package_name }}"
        state: present
```

VARIABLE PRECEDENCE

AVOID DEFINING THE VARIABLE "X" IN 47 PLACES

AND THEN ASK THE QUESTION "WHICH X GETS USED".

1. Extra vars - always win
2. task vars (only for the specific task)
3. block vars
4. Role and inculde vars
5. Vars created with set_fact
6. Vars created with register task directive
7. Play vars_files
8. Play vars_prompt
9. Play vars
10. Host facts
11. Playbook host_vars
12. Playbook group_vars
13. Inventory host_vars
14. Inventory group_vars
15. Inventory vars
16. Role defaults

LOOPS

```yaml
---
- hosts: webapp
  gather_facts: no
  tasks:
    - name: loop over a list
      debug:
        msg: "An item: {{ item }}"
      with_items:
        - 1
        - [2,3]
        - 4

    - name: loop over a list of hashes
      debug:
        msg: "{{ item.name }} is {{ item.job }} with index {{ my_index }}"
      loop:
        - { name: Alice, job: Developer }
        - { name: tabutha, job: Devops }
      loop_control:
        index_var: my_index

    - name: loop over a nested lists
      debug:
        msg: "{{ item[0] }} is {{ item[1] }}"
      with_nested:
        - ['Vlad','Eugen']
        - ['devops','developer','pm']

    - name: Show all the hosts in the inventory
      ansible.builtin.debug:
        msg: "{{ item }}"
      loop: "{{ groups['all'] }}"

    - name: Show all the hosts in the current play
      ansible.builtin.debug:
        msg: "{{ item }}"
      loop: "{{ ansible_play_batch }}"
```

Loops allow you to repeat a task multiple times

DIRECTORY LAYOUT

```
staging                 # inventory file for staging environment

group_vars/
  group1.yml            # here we assign variables to particular groups
  group2.yml
host_vars/
  hostname1.yml         # here we assign variables to particular systems
  hostname2.yml

library/                # if any custom modules, put them here (optional)
module_utils/           # if any custom module_utils to support modules, put them here (optional)
filter_plugins/         # if any custom filter plugins, put them here (optional)

site.yml                # master playbook
webservers.yml          # playbook for webserver tier
dbservers.yml           # playbook for dbserver tier
```

MASTER PLAYBOOK

```yaml
---
# file: site.yml
- import_playbook: webservers.yml
- import_playbook: dbservers.yml
```

THE PLAYBOOK

A PLAYBOOK IS PARSED BEFORE BEING RUN. ANY INCLUDED PLAYBOOKS NEED TO EXIST AT THE TIME OF PARSING.

1. Operations within a play:
2. Variable loading
3. Fact Gathering
4. The pre_task execution
5. Handlers notifed from pre_tasks
6. Roles execution
7. Tasks execution
8. Handlers notified from roles or tasks execution
9. The post_tasks execution
10. Handlers notified from post_tasks

EXECUTION STRATEGIES

LINEAR

All hosts for a given play execute first task.

Once all complete the task, next one is executed

FREE

Host completes tasks on it’s own

IDEMPOTENCE

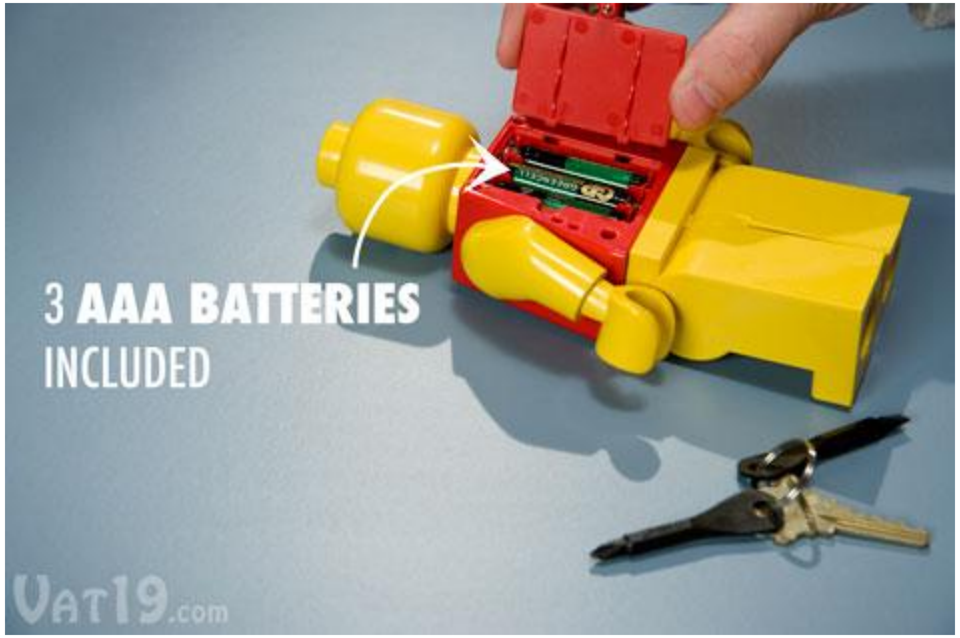
THE PROPERTY OF CERTAIN OPERATIONS THAT THEY CAN BE APPLIED MULTIPLE TIMES WITHOUT CHANGING THE RESULT BEYOND THE INITIAL APPLICATION.

BATTERIES INCLUDED

MODULES

- CLOUD MODULES
- FILES MODULES
- MONITORING MODULES

…



HTTP://DOCS.ANSIBLE.COM/ANSIBLE/LATEST/MODULES/MODULES_BY_CATEGORY.HTML

SHELL MODULE

```
- name: remove a text file in $HOME only if it already exists
  shell: rm $HOME/test_file.txt
  args:
    removes: "$HOME/test_file.txt"
```

CONDITIONALS

```yaml
---
- name: demo the template
  hosts: localhost
  gather_facts: false
  vars:
    setting: a_val
    feature:
    enabled: true
    another_setting: b_val
  tasks:
    - name: pause with render
    pause:
      prompt: "{{ lookup('template', 'demo.j2') }}"
```

DATA MANIPULATION | FILTERS

my_word : “SOME TEXT IN CAPS”
answers: “no, no yes, no, yes”
answers: “no, NO yes, NO, Yes”

{{ my_word | lower }}
{{ answers | replace('no', 'yes') }}
{{ answers | replace('no', 'yes') | lower }}

useful filters
•default
•count
•random
•related to task status | changed
•basenane ("{{ '/var/log/nova/nova-api.log' | basename }}") = nova-api.log
•Dirname = '/var/log/nova/’
•b64decode or encode

USEFUL LINKS

https://docs.ansible.com/
https://docs.ansible.com/ansible/latest/user_guide/intro_getting_started.html
https://docs.ansible.com/ansible/latest/user_guide/intro_inventory.html
https://docs.ansible.com/ansible/latest/user_guide/playbooks_best_practices.html
https://docs.ansible.com/ansible/latest/user_guide/playbooks_filters.html

ansible introduction youtube playlist

good book for advancing in ansible : Mastering Ansible - Third Edition

EUGEN BUGA
DEVOPS ENGINEER
eugen.buga@endava.com